

Week 5 Lab Reflection

Decorator + Factory Patterns

Design Reflection

1. Why We Used the Priced Interface

We decided to add a Priced interface to make it easier to get the final price of a product, whether it's just a normal coffee or one with a bunch of extras.

Basically, the Product interface still handles the base price, and Priced takes care of the final price after all the add-ons are added. This made things a lot cleaner — we don't need to worry about which add-ons are there, we just ask the product for its price and it gives us the right total.

It also means both simple products and decorated ones work in the same way, which makes the rest of the code simpler.

-

2. Keeping the Code Open for Extension

One of the main goals was following the Open/Closed Principle — meaning we can add new features without changing the old ones.

For example, if we want to add something like Whipped Cream, we can just make a new decorator class for it, tell the factory about it, and everything works straight away. No need to touch any existing code.

This is a big improvement compared to before, when a new feature meant editing a bunch of different files and hoping nothing broke.

-

3. Adding a New Add-On (Example)

Let's say next week we want to add Vanilla Flavor. It's now super easy:

1. Make a new decorator for it that adds the extra price.
2. Add it to the factory so it knows what "VAN" means.
3. Test it once to make sure it works.

That's it - no rewriting or messy edits anywhere else. It just plugs right in.

-

4. The Factory's Role

The factory kind of acts like the café's "menu." It knows how to build each drink and what each token (like "SHOT" or "OAT") means.

Even though we do have to add one small line when creating a new add-on, it's fine because it keeps everything organized in one place. The rest of the code doesn't even know how the drinks are built — it just asks the factory.

This makes the system way easier to maintain and understand.

-

Summary

Using the Decorator and Factory patterns made a huge difference.

- The Decorator pattern lets us stack add-ons however we want.
- The Factory pattern makes product creation simple and consistent.
- The Priced interface keeps everything connected and makes pricing easy.

Test Results

Test Class	Test Results
✓ DecoratorTests (com.cafepos.decorator) 27 ms ✓ syrup_decorator_adds_correct_surcharge() 22 ms ✓ oatMilk_decorator_adds_correct_surcharge() 1 ms ✓ factory_creates_large_latte() 2 ms ✓ decorator_stacks() ✓ order_uses_decorated_price() 1 ms ✓ factory_parses_recipe() ✓ decorator_single_addon() ✓ factory_handles_case_insensitive_input() ✓ factory_rejects_invalid_addon() 1 ms ✓ factory_rejects_invalid_base() ✓ sizeLarge_decorator_adds_correct_surcharge() ✓ multiple_decorators_price_is_commutative()	✓ 12 tests passed 12 tests total, 27 ms /Users/islamkastero/Library/Java/JavaVirtualMachine Process finished with exit code 0

Test Class	Test Results
✓ FactoryVsManualTest (com.cafepos.decorator) 24 ms ✓ factory_and_manual_latte_are_identical() 23 ms ✓ factory_and_manual_produce_equal_order() 1 ms ✓ factory_and_manual_complex_drink_are_identical() ✓ factory_and_manual_produce_identical_drinks()	✓ 4 tests passed 4 tests total, 24 ms /Users/islamkastero/Library/Java/JavaVirtualMachine Process finished with exit code 0